Object Oriented Programming (CSEG1044)
Unit 03 – Lecture 04 Notes
Java I/O Streams + FileReader/FileWriter

Tofik Ali

February 27, 2026

Contents

1 Syllabus Alignment (Unit 03) **2**

2 Prerequisites **2**

3 Learning Outcomes **2**

4 Lecture Overview **2**

5 Key Terms (Quick) **2**

6 Detailed Notes **3**

6.1 Stream model (big picture) . . . 3
6.2 Byte streams vs character streams . . . 3
6.3 Text file I/O with `FileReader`/`FileWriter` . . . 3
6.4 Reading patterns (three common styles) . . . 4
6.5 Exception-safe I/O: `try-with-resources` . . . 4
6.6 Paths and a common beginner mistake . . . 4
6.7 Overwrite vs append (writing) . . . 4

7 Worked Example (tutorial): read marks and write a report **5**

8 Mini tutorial: copy a text file **6**

9 In-Class Exercises (with brief solutions) **7**

10 Self-Study Practice (with hints) **7**

11 Summary (what to remember) **7**

12 Lab Alignment (recommended) **7**

13 Checkpoint Questions (with sample answers) **7**

1 Syllabus Alignment (Unit 03)

Unit 03: Java I/O streams; `FileReader`, `FileWriter`

2 Prerequisites

You should already know:

- basic exception handling (`try/catch`),
- loops and string trimming/parsing,
- how to run a Java program from an IDE or terminal.

3 Learning Outcomes

After this lecture, you should be able to:

- Explain the stream model and choose between byte streams and character streams.
- Read and write text files using `FileReader`/`FileWriter` with buffering.
- Write exception-safe I/O code using `try-with-resources` and meaningful error messages.

4 Lecture Overview

In almost every real application you must read/write data:

- read configuration,
- read input files (CSV/text),
- write reports/logs,
- store results for later use.

Java provides a stream-based I/O model. In this lecture we focus on text files using `FileReader`/`FileWriter` and buffering, and we learn how to avoid the most common beginner mistakes (wrong path, not closing files, confusing exceptions).

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed February 27, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed February 27, 2026).

5 Key Terms (Quick)

- **Stream**: sequence of data flowing from a source to a destination.
- **Input stream** vs **output stream**: read vs write.
- **Byte stream**: reads/writes raw bytes (`InputStream`/`OutputStream`).

- **Character stream**: reads/writes characters (`Reader`/`Writer`).
- **Buffering**: collecting data in memory to reduce expensive I/O operations.
- **try-with-resources**: `try(...)` that auto-closes resources.

6 Detailed Notes

6.1 Stream model (big picture)

I/O in Java is organized as streams:

- A **source** (keyboard, file, network),
- A **stream object** (API you use),
- A **destination** (screen, file, network).

Streams are **one-directional**: you create an input stream to read and an output stream to write. You can also **chain** streams: for example, a file reader wrapped by a buffered reader.

6.2 Byte streams vs character streams

Byte streams (`InputStream`/`OutputStream`) are used for binary data:

- images, PDFs, audio/video, serialized binary formats.

Character streams (`Reader`/`Writer`) are used for text:

- plain text files, logs, reading lines of text.

Rule of thumb. Use `Reader/Writer` for text; use `InputStream/OutputStream` for binary.

6.3 Text file I/O with `FileReader`/`FileWriter`

Basic idea:

- `FileReader` reads characters from a file.
- `FileWriter` writes characters to a file.
- These can be wrapped in buffered versions for performance.

Important note (encoding). `FileReader/FileWriter` use the platform default charset. For most course examples this is fine, but in real projects you often choose a charset explicitly (UTF-8).

Buffering:

- `BufferedReader` provides `readLine()` and reads in chunks.
- `BufferedWriter` provides efficient writing and `newLine()`.

6.4 Reading patterns (three common styles)

1) Read character-by-character (rarely used in app code):

- simple idea, but slower and harder to manage.

2) Read line-by-line (most common for text files):

- use `BufferedReader.readLine()`.

3) Read tokens/CSV (usually needs parsing/splitting after reading lines):

- read line → split by delimiter → parse fields.

6.5 Exception-safe I/O: `try-with-resources`

File I/O can fail for many reasons: wrong path, missing file, permissions. Also, resources must be closed.

Use:

- `try (BufferedReader br = ...){...}` to auto-close even if exception occurs.

This reduces resource leaks and makes code cleaner.

6.6 Paths and a common beginner mistake

The JVM reads files relative to the "working directory" (where you run `java`). If the file is "not found":

- confirm the path,
- print the working directory (or use absolute path),
- ensure file extension and capitalization are correct.

Quick debug snippet (print working directory).

```
System.out.println(new java.io.File(".").getAbsolutePath());
```

Windows path gotcha (Java strings). In Java string literals, backslash is an escape character:

- wrong: `"D:\data\marks.txt"` (contains escape sequences)
- correct: `"D:\\data\\marks.txt"` or `"D:/data/marks.txt"`

6.7 Overwrite vs append (writing)

`FileWriter(path)` overwrites the file by default. To append:

```
new FileWriter("log.txt", true); // append=true
```

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed February 27, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed February 27, 2026).

7 Worked Example (tutorial): read marks and write a report

Goal

Read integer marks from `marks.txt` (one mark per line), compute sum/avg/max, and write a report to `report.txt`.

Input file example

```
78
91
65

88
```

Blank lines are allowed; we will ignore them.

Code

```java
import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;

public class MarksReport {
    public static void main(String[] args) {
        String inPath = "marks.txt";
        String outPath = "report.txt";

        int sum = 0, max = Integer.MIN_VALUE, count = 0;

        try (BufferedReader br = new BufferedReader(new FileReader(inPath))) {
            String line;
            while ((line = br.readLine()) != null) {
                line = line.trim();
                if (line.isEmpty()) continue;

                int x = Integer.parseInt(line);
                sum += x;
                if (x > max) max = x;
                count++;
            }
        } catch (IOException e) {
            System.out.println("I/O error: " + e.getMessage());
            return;
        } catch (NumberFormatException e) {
            System.out.println("Invalid number in input file: " + e.getMessage());
            return;
        }

        double avg = (count == 0) ? 0.0 : (sum / (double) count);

        try (BufferedWriter bw = new BufferedWriter(new FileWriter(outPath))) {
```

```
            bw.write("Count: " + count);
            bw.newLine();
            bw.write("Sum: " + sum);
            bw.newLine();
            bw.write("Max: " + max);
            bw.newLine();
            bw.write(String.format("Avg: %.2f", avg));
            bw.newLine();
        } catch (IOException e) {
            System.out.println("Write error: " + e.getMessage());
        }
    }
}
```

What to notice

- `try-with-resources` closes readers/writers automatically.
- `BufferedReader.readLine()` makes line-by-line reading easy.
- We handle both I/O errors and invalid numbers separately (better messages).
- We use `String.format` for formatted average.

Common improvements (good practice)

- Validate mark range (0–100) and count invalid rows.
- Show a clearer message when the file is missing (path help).
- Write results in a CSV format if needed.

8 Mini tutorial: copy a text file

Copying is a great exercise because it uses both reading and writing.

```
import java.io.*;

public class CopyTextFile {
    public static void main(String[] args) throws IOException {
        String src = "input.txt";
        String dst = "output.txt";

        try (BufferedReader br = new BufferedReader(new FileReader(src));
             BufferedWriter bw = new BufferedWriter(new FileWriter(dst))) {

            String line;
            while ((line = br.readLine()) != null) {
                bw.write(line);
                bw.newLine();
            }
        }
    }
```

```
}
```

9 In-Class Exercises (with brief solutions)

1. Which should be used for text files: byte stream or character stream?

   Answer: character stream (`Reader/Writer`).

2. Why do we use buffering?

   Answer: it reduces the number of expensive disk I/O calls by reading/writing in chunks.

3. If a file is not found, name one common cause.

   Answer: wrong working directory / wrong path.

10 Self-Study Practice (with hints)

1. Write a program that copies a text file (read from one, write to another) using buffered streams. **Hint:** the copy tutorial above is the solution pattern.

2. Write a program that counts lines and words in a file. **Hint:** count lines with `readLine`; count words with `split("`

   `s+")`.

3. Modify the worked example to ignore marks outside 0–100 and count how many were invalid. **Hint:** after parsing, check range and continue.

11 Summary (what to remember)

- Streams model: input vs output; chain streams for features like buffering.
- Use `Reader/Writer` for text and buffer with `BufferedReader/BufferedWriter`.
- Use `try-with-resources` for reliable closing.
- Most file bugs come from wrong paths/working directory: print it and verify.
- Handle `IOException` separately from parse/validation exceptions.

12 Lab Alignment (recommended)

- Exp-13: File Handling using I/O streams (based on this lecture)

13 Checkpoint Questions (with sample answers)

Q1. Why is `try-with-resources` preferred?

Sample answer: It automatically closes resources even if an exception occurs, preventing resource leaks and reducing boilerplate code.

Q2. When do you choose byte streams instead of character streams?

Sample answer: Use byte streams for binary data like images or PDFs. Use character streams for text data.